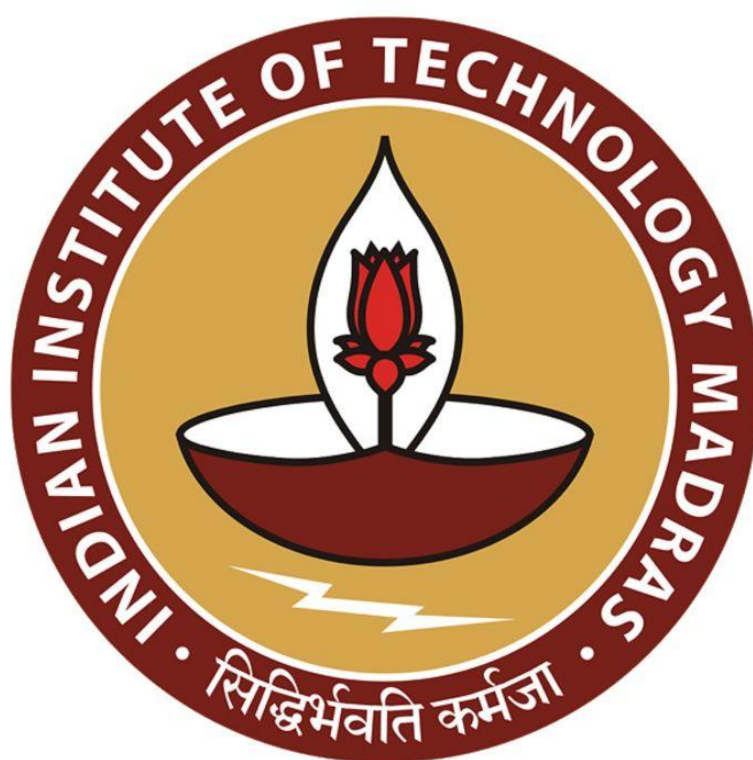




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun RajKumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Introduction to Decision Trees

Hello. So, far in supervised learning, we have looked at binary classification. And we have looked at the problem of K nearest neighbors, and the algorithm called K nearest neighbors to solve the problem of binary classification, we identified an important problem with K nearest neighbors, which is the fact that K nearest neighbors does not learn any model from data.

So, we asked the question, can we come up with algorithms, which given a dataset learn some interesting model from this data, and once the model is learned, we can throw away the dataset and only use the model to make predictions on test data points.

Today, now, what we are going to see is such an algorithm, which is a simple algorithm, very popular algorithm, and it is it kind of is a base algorithm for several complicated algorithms that we will see later on as well.

(Refer Slide Time: 01:02)

DECISION TREES

INPUT: Dataset $\{ (x_1, y_1), \dots, (x_n, y_n) \}$

$x_i \in \mathbb{R}^d$
 $y_i \in \{+1, -1\}$

OUTPUT: DECISION TREE

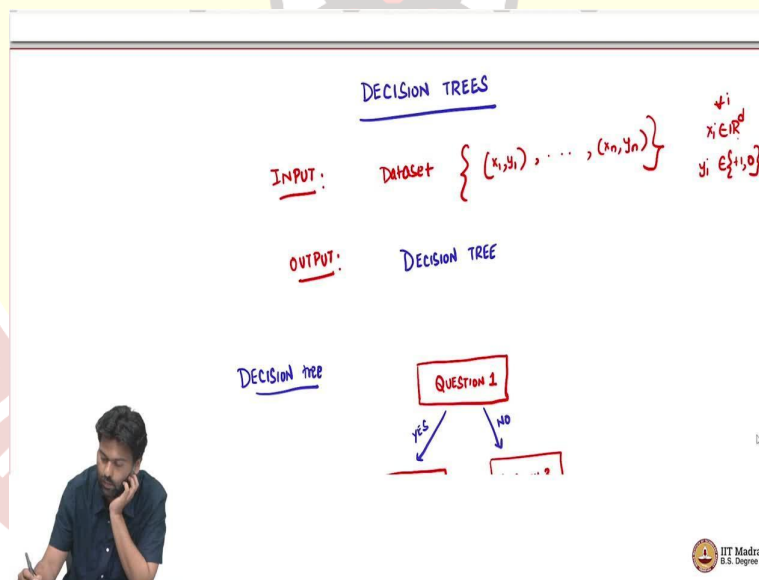
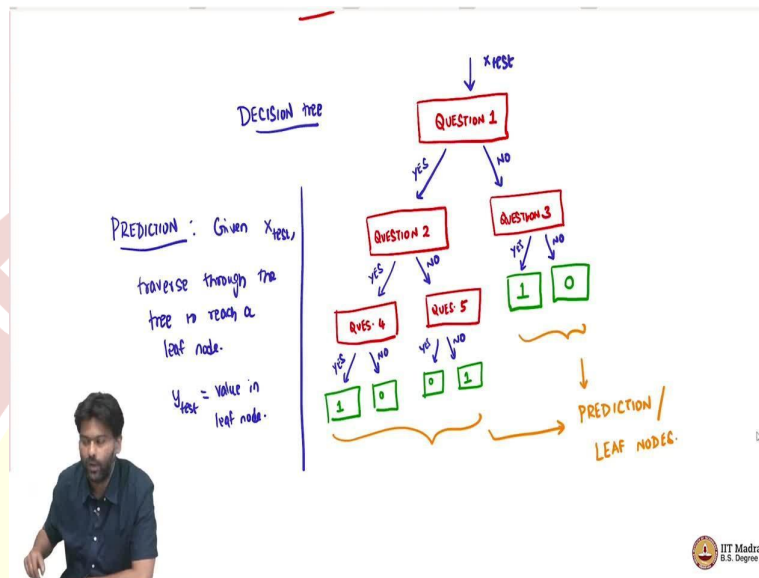
16

IIIT Madras
B.S. Degree

And this algorithm is what is called as the decision tree algorithm. What is the input to this algorithm? As usual, the input is a data set, which is a training data set, which looks like as usual, (x_1, y_1) , dot, dot, dot, (x_n, y_n) where x_i is in d dimension for all data points. And y_i is in $\{+1, -1\}$, this is a binary classification problem.

Now, what is going to be the output of this algorithm is? You take the dataset and then output a decision tree. And this decision tree is going to be used to make future computation future predictions. So, what is a decision tree?

(Refer Slide Time: 01:58)



A decision tree is going to look as follows. How is the decision tree going to look like? It is a tree, a tree in the sense of computer science, tree is going to look like following. So, it is a binary tree. And it is going to have some nodes, it is a root tree, it is going to have a root node. And then you are going to have some internal nodes, and then you are going to have some leaf nodes, that is our tree typically is.

So, that in every node, you are going to ask a question. Let us say this is question 1. So, this is a yes or no question, which means it has an answer to this question can be either yes or no? And now, if the answer is yes, then you ask a different question, question 2, if the answer is no, then perhaps you ask a different question, question 3, and so on. Now, what might happen is, if the question is, question 2 can have again 2 answers, yes or no. And now if it can still go on, so maybe you increase the number of questions, maybe question 4, I am just giving you an example, question 5.

Whereas question 3 has two answers, yes or no. But then you do not further ask any questions. You make a decision here. The green boxes are decision boxes. And what might be the decision? The decision in this case is a 1 and in this case, let us say some -1, or for this algorithm, let us say it is 0. So, we can simply does not really matter, we can, but I am just going to use 0 to indicate negative samples.

Whereas in the other side, you ask extra 2 questions. And then may, based on that, you might make a decision, this might be yes or no, this is a yes no. And now you make the decisions maybe here it is 1, here it is 0, here it is 0 here it is 1. So, these nodes, these guys are called as prediction or leaf nodes. So, this is the tree.

So, this is we will talk more about what exactly are these questions and so on. But this is the structure of the tree. It is a binary tree. It is gone every root, every node internal node, all the red boxes are internal nodes, every internal node is has a question associated with it. And then there are 2 answers to it. And the green nodes are the leaf nodes of the prediction nodes. So, now how would we predict? The prediction is easy. The prediction is going to be as follows.

So, somehow, let us say somebody gives you this tree that they have constructed from the data set. Now you are given a new test data point, let us say x_{test} . Now, all you do is traverse through the tree to reach a leaf node. Now, y_{test} is the value in the leaf node. What do I mean by that? I mean that, you pass the test data point x_{test} to this tree and ask the series of questions and then if the questions answer for the x_{test} data points that you pass as no, then you go to the right side if it is yes, you go to the left side, which means that you are tracing out a path in this tree. And every path ends on a leaf node. And the leaf node has a corresponding 1 or 0 associated with it. And then that would be the label that we predict.

So, basically, what we have done is, we are creating a series of questions from our data set. And these questions are enough, we say to make predictions of test data points. So, the model here, which was lacking in a K nearest neighbor problem, the model is the tree. Once I have this tree, I do not need the data set anymore. This tree encodes all the information that I need for prediction that was available in my data set.

Once I have this, I can throw away the data set and work just with the tree, which is enough to make future predictions for any given data point. So, that is why this is a powerful thing. Because the prediction is fast. So, you just have to traverse through the trees, answer a bunch of questions, and then outcomes the answer.

So, now the questions that we have to ask is, what are these questions? So, how do I mean how do I build this tree is one thing, but even before we build the tree, we need to ask ourselves, what are we? What type of questions are we asking? So, that we can think about how to build the tree in an efficient way? So, what is the question?

(Refer Slide Time: 07:21)

QUESTION : A question is a (feature, value) pair.

Eg: height $\leq$ 180cm ?
(f3) θ

How to measure "goodness" of a question?

IIT Madras
B.S. Degree

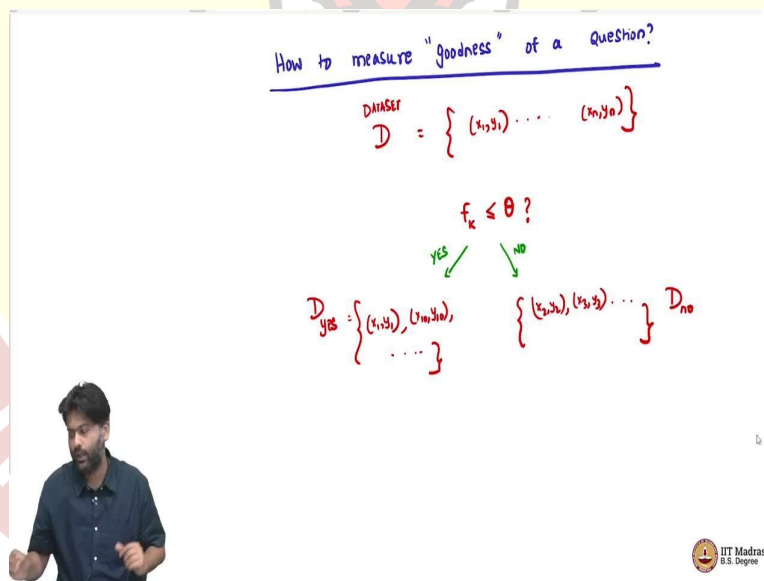
Question? In a binary tree, in a decision tree, a question you can think of a question as a feature value pair. What does that mean? An example would be feature is height, let us say height and the value is 180 centimeters. Now, the question that we will create out of this is as follows; is the height $\leq$ 180 centimeters? Maybe this is feature 3, maybe this is some value θ . So, in general, it is some feature and then we are asking is some feature less than or equal to some value? This is the type of questions that we are going to restrict our tree to have.

So, once we know what kind of questions we are going to ask, then we need to build this tree somehow from the dataset. Now, which means we have to, what we are going to do is? We are going to greedily ask questions. So, we have to first somehow find what is the best question to ask? And then we will ask, we will later on try to expand this tree. So, which means we need to somehow have a measure of how good a question is? So, what we need immediately then is? We have to ask ourselves how to measure goodness of question?

So, what might be a good way? So, this might be a good time to pause and think, how do you, how can you measure goodness of a question? So, just to just to, point you in the, perhaps a useful direction to think about, instead ask the question, what would be the property of the best possible question? So, I have a data set. And my goal is to do prediction well.

Now, what would be really good question? So, what property should that question have? let us ask this ourselves.

(Refer Slide Time: 09:50)



Now, what is a question do to our data set? So, remember, we have this data set, let us call this D . So, data set, which is just a bunch of data points of this form (x_1, y_1) , dot dot dot, (x_n, y_n) . And now we ask a question, let us take one particular question, some feature, let us say height is less than or equal to some value, let us say 100, 120 centimeters, 130 centimeters, whatever. So, we ask this question, of course, this is a yes or no question.

Now, there are, I can ask this question to every data point in my data set. So, and there are two answers yes and no. Now, some data points will have features where the height < 180 centimeters, and some data points will have the feature height > 180 centimeters. So, which means that this question is dividing my dataset into two parts.

So, there is one part which is D yes, I am going to call this the D yes part, which is just a dataset with maybe the first point answered yes, maybe the 10th point answered yes, (x_{10}, y_{10}) dot dot dot some bunch of points. So, it is a yes or no question. So, every point either answers yes or no. So, which means there is a D no also, which has the remaining set of points (x_2, y_2) , (x_3, y_3) , dot, dot, dot, and so on. So, it is, you are separating, you are partitioning your data set into two parts, where one part answered yes to this question the other part answered no to this question.

Now, what would be an ideal question? A really good question should have what property? The best possible question. It should have the property that I should be able to make prediction by just asking this question. Which means when will I be able to make a solid prediction by asking just this question? In my training data, if I asked this question, if height < 180 centimeters, and I observed that every data point which answers yes, has the corresponding label as +1 and every data point that answers no has the corresponding label -1 or the other way around.

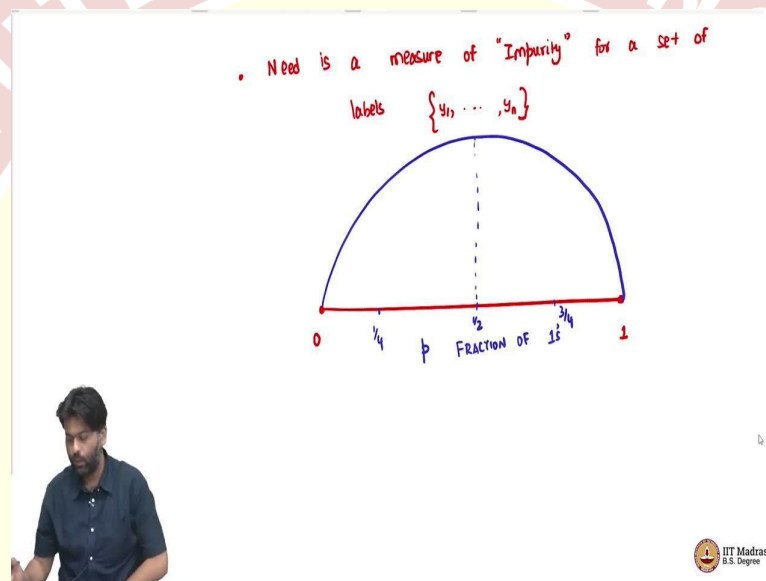
Every data point that answers yes has the corresponding label -1 and every data point that answers no has the corresponding label +1, then I know that this is a super good question, because by asking the single question, I am able to distinguish the +1 from the -1. Everybody answered, yes, have the same label and everybody answers no have the same label, then this is a very good question. Because this one question is enough to, classify whether the data point is +1 or -1 in my training set, which means that I will hope to do the same thing for my test data point also.

If the test data point comes, I will just look at the height. If it is < 180 centimeters, then I will say maybe +1, and otherwise, I will say -1, depending on what the dataset says. Now, that is an ideal world. So, where you have a single super good question, which is enough to say, divide my data set into two parts, where the labels of one side is the same, and the labels of the other side is also the same.

But in reality, such a question may not exist, there may not be a single question that you can ask such that you can divide your data sets into pure sets. So, when I say pure, it means that the labels on each of the sides are pure, where exactly the same. Such pure questions may not exist. So, which means we somehow have to measure how good a question is, by somehow capturing a notion of impurity for a given set of labels. So, I divided into two parts.

Now, I need to see how impure each of these parts are. So, or how pure each of these parts are? So, I somehow have to measure this So, which means I know how to measure.

(Refer Slide Time: 13:57)



So, need, what we need is a measure of impurity for a set of labels. Let us say I give you a bunch of labels So, y_1 to y_n , or y_k . So, the data set can be of any size. So, after you ask the question, only k points have answered yes. I need to I need to associate a measure of impurity. These y labels are either, $+1$ or 0 . So, we can, for instance, look at the fraction of ones. So, we can, we can see we can look at the fraction of ones in my set of labels.

I give you a bunch of labels. The question is how impure is this bunch of labels? I can compute how many 1 s' are there or the fraction of 1 s'. And that fraction could be any value between 0 to 1 . So, this is the fraction of 1 s', fraction of 1 s' in my set, let us call this value p . p can be any value between 0 to 1 , if p is 1 , then that means that all my labels are 1 , if p is 0 , that means all my labels are 0 , there is no 1 s' at all.

Now, in both these cases, there is no impurity because the labels are clean. So, it is either all 1 s' or all 0 s, both are good for us. But the worst case would happen when p is half. So, when

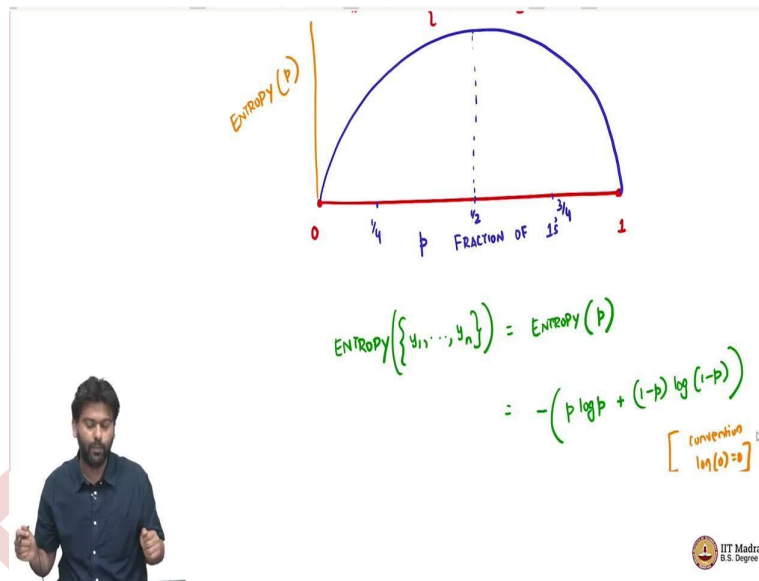
you have a bunch of labels, where 50 percent of them are positive 50 percent of them are zero, then you are still very uncertain about this set of data points. So, because you will not be able to say anything concrete about the label.

So, the measure of impurity should be highest at 0.5, and it should be 0 at 0 and 1. So, also, if I take let us say, a measure of if p is, let us say, quarter, So, we know it is, it should p cut half. Now, if it is at quarter, that means that 25 percent there is 1 and 75 percent there is 0. Now, this situation is as impure, as if you flip it, that there are 75 percent 1 and 25 percent 0. So, there is no nothing more sacrosanct about 1 than 0.

So, which means that if you have 25 percent 1s' and 75 percent 0s', it is as impure as 75 percent 1s' and 25 percent 0s', your uncertainties is in some sense is the same about what label this class below, this set of data points. If I had to make a prediction, what would I predict is still confusing. So, it is a same amount of confusion you have. So, which means that one fourth should have the same amount of impurity as three fourths, So, which means any, any value, λ should have the same impurity as $1 - \lambda$, it should be basically symmetric about half.

So, one function which has this property, which looks like this, So, the function looks like this, it peaks at 0.5. And I have drawn it slightly skewed, but then it is symmetric about half. So, the values at one fourth and three fourths will be the same, it will be 0 at the ends. So, one such function is what is called as the entropy function. So, we could use many functions which have this property.

(Refer Slide Time: 17:33)



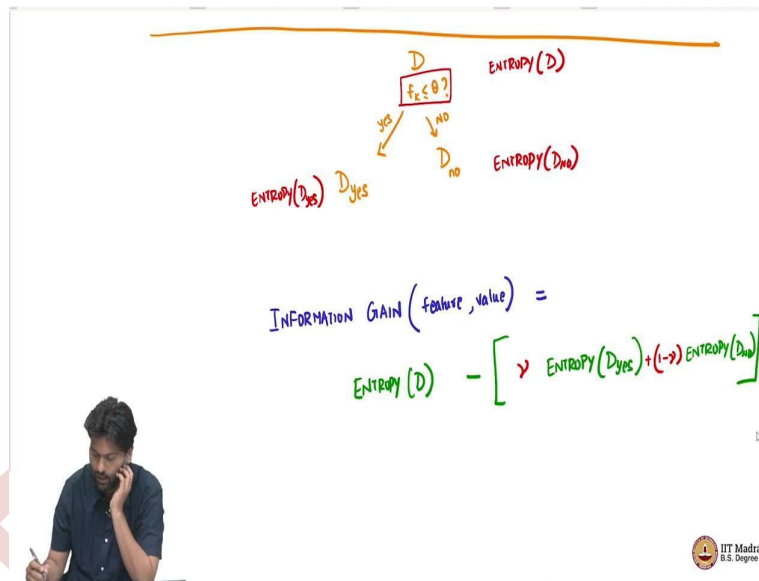
One popular function that people use is the entropy. Entropy, you think of entropy as a bunch of labels, that is what we are trying to find out some bunch of labels is just the entropy of the fraction of 1s' because that is what determines how pure or impure this set of labels is? Is defined as the following. It is defined as $-(p \log p + (1-p) \log (1-p))$. With the understanding that, the convention says that log of 0, you should treat as 0.

So, if this one possible function, which captures our notion of impurity, and basically what you would get, if you plot here is just the entropy of p, what I am plotting on the y axis is just entropy of p, it will collect this, it is a concave function, which looks like this. So, this is a reasonable measure of, of impurity. Of course, there are some information theoretic reasons why this, this makes sense and so on, we will not get into the information theoretic.

In fact, it has, it has underpinnings in statistical physics, from thermodynamics, and so on. That is where the name entropy comes from and then which was borrowed by information theory by Shannon. And then, for our purposes, we should we can just think of measuring impurity. So, some function that measures impurity, that is what we are, we care about.

So, what we now have is some way to measure how impure a bunch of labels are. So, now we have to use this somehow, to come up with a goodness for a question. So, that is our goal So, given a question, how good is this question?

(Refer Slide Time: 19:32)



Now, remember, we had a data set D . And then we asked a question. So, we have a data set D and then we ask a question, $f(k \leq \theta)$ it divided my data set into two parts D_{yes} and D_{no} . This is the yes guys, this is the no guys this is D_{no} .

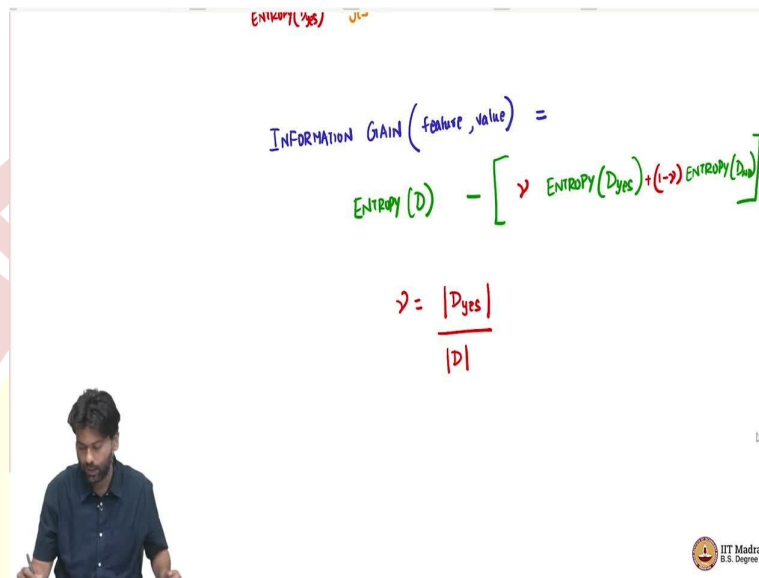
Now, what I am saying is I have a measure to capture impurity for a bunch of labels. Now, which means I have I can capture entropy of D , which means I look at the labels in the original data point dataset, and that will give me some entropy. I can look at entropy of D_{yes} , so when I say D_{yes} , I mean that the fraction of labels which are 1 in D_{yes} is what I am computing an entropy for. And I can look at entropy for D_{no} , so D_{no} . So, these are three different numbers that I can calculate.

So, it tells us how impure D is? How impure D_{yes} is and how impure D_{no} is? So, for every question, I have these three numbers that immediately come out, So, the moment I put down an equation $f(k < \theta)$, I get D_{yes} , and D_{no} from which I get entropy of each of these, and I of course have the original entropy of the dataset itself. So, but then there are three numbers.

We need to associate a single number to measure goodness of a question. So, how can we convert these three numbers in a natural way to associate a goodness for question $f(k \leq \theta)$? one way to do this is an and perhaps the most natural way to do this is the following. You look at what is usually called as again, from information theoretical underpinnings is called information gain of a feature comma value, so of a question which is feature comma value pair, can be defined as you know how much impurity I originally had, which is entropy of the original data set D .

Now, I have entropy of D yes and I have entropy of D no. Now, I am going to subtract this impurity from some combination of the impurity of D yes and D no. Now what combination I am going to use? I am going to use some combination that depends on how many data points fell on each side.

(Refer Slide Time: 22:22)



ENTROPY(Dyes) Dno

$$\text{INFORMATION GAIN}(\text{feature, value}) =$$

$$\text{ENTROPY}(D) - \left[\lambda \text{ ENTROPY}(D_{\text{yes}}) + (1-\lambda) \text{ ENTROPY}(D_{\text{no}}) \right]$$

$$\lambda = \frac{|D_{\text{yes}}|}{|D|}$$

IIT Madras
B.S. Degree

So, let us say λ is the fraction of data points that went to the D yes side and fraction of data points that went into the D no side is $1 - \lambda$ so λ , sorry, λ . So, I am using not λ , λ , λ is just, you know, how many guys went to D yes cardinality of the set D yes, by cardinality of the set D.

Now, why am I combining this using λ ? So, the reason is, So, let us say you ask a question, and then you divide it into D yes, and D yes had 80 percent 1s'. So, now, if D yes had 10,000 data points out of which 80 percent were 1s', which means 8000 points were 1s'. Now, that should kind of give us more confidence about the label that we can predict after asking this question, then saying you had 100 data points on this site out of which 80 were 1.

So, in some sense, what we are saying is that we should also measure by how many data points fell on this side, in addition to what is the impurity? Because the impurity is not going to take into account the number of data points, it is only going to look at the fraction of 1s' or 0s' in the labels, it is oblivious to the number of data points, so both two datasets where you had 10,000 points with 9999 of them as 1 or 9900 as 1 is same amount of impure as per entropy as a data set which had 99 out of 100.

So, but then you the same question is able to classify more points. So, if you had 9900 out of 10,000, So, you somehow have to weigh that by then fraction of points that fell on each side. And that fraction is γ . So, basically, what we are saying is that initially, you had some impurity, and how much impurity gets removed by asking this question. So, by subtract, that is why we subtract out the weighted average of entropies of D_{yes} and D_{no} weighted by the amount of fraction of data points that fell on each side from the original impurity.

Now, if the question is super good, then both the sides would become pure just by after asking this question, which means that the entropy of D_{yes} and D_{no} will both be 0. So, which means the amount of impurity drop would be the entire impurity that we had in the origin D . Which means your entropy of $D - 0$ which means your gain and information or the loss and impurity would exactly be the amount of impurity that you originally had.

So, the more the impurity that drops, the more information you gain. So, because you are able to make prediction better that so information regarding prediction becomes better. And so this is also called as an information gain. So, so now we somehow have a notion of goodness of a question. So, the next natural thing to do is how can I convert this into an algorithm? So, we will put down that algorithm next.

